College of Engineering, Construction and Living Sciences
Bachelor of Information Technology
ID511001: Programming 2
Level 5, Credits 15

Project 2

Assessment Overview

In this **individual** assessment, you will design and develop a **Windows Forms Application** using **C#**.

Learning Outcomes

At the successful completion of this course, learners will be able to:

1. Build interactive, event-driven GUI applications using pre-built components.
2. Declare and implement user-defined classes using encapsulation, inheritance and polymorphism.

Assessments

Assessment	Weighting	Due Date	Learning Outcomes
Project 1	25%	Sunday 9th November at 11.59 PM	1, 2
Project 2	35%	Sunday 14th September at 11.59 PM	1, 2
Theory Examination	30%	Wednesday 8th and Thursday 9th October from 10.00 AM - 11.45 AM	1, 2
Classroom Tasks	10%	Sunday 12th October at 11.59 PM	1, 2

Conditions of Assessment

You will complete this assessment mostly during your learner-managed time. However, there will be time during class to discuss the requirements and your progress on this assessment. This assessment will need to be completed by **Sunday 14 September** at **11.59 PM**.

Pass Criteria

This assessment is criterion-referenced (CRA) with a cumulative pass mark of **50%** over all assessments in **ID511001: Programming 2**.

Authenticity

All parts of your submitted assessment **must** be completely your work. Do your best to complete this assessment without using AI tools. You need to demonstrate to the course lecturer that you can meet the learning outcome for this assessment.

Learning to use AI tools is an important skill. While AI tools are powerful, you **must** be aware of the following:

- If you provide an AI tool with a prompt that is not refined enough, it may generate a not-so-useful response
- Do not trust the AI tool's responses blindly. You **must** still use your judgement and may need to do additional research to determine if the response is correct
- Acknowledge what AI tool you have used. In the assessment's repository **README.md** file, please include what prompt(s) you provided to the AI tool and how you used the response(s) to help you with your work

It also applies to code snippets retrieved from **StackOverflow** and **GitHub**.

Failure to do this may result in a mark of **zero** for this assessment.

Policy on Submissions, Extensions, Resubmissions and Resits

The school's process concerning submissions, extensions, resubmissions and resits complies with **Otago Polytechnic** policies. Learners can view policies on the **Otago Polytechnic** website located at https://www.op.ac.nz/about-us/governance-and-management/policies.

Submission

You **must** submit all application files via **GitHub Classroom**. Here is the URL to the repository you will use for your submission – https://classroom.github.com/a/Xg7esDmf. If you do not have not one, create a **.gitignore**

and add the ignored files in this resource - https://raw.githubusercontent.com/github/gitignore/main/VisualStudio.gitignore.

Create a branch called **project-2**. The latest application files in the **project-2** branch will be used to mark against the marking rubric. Please test before you submit. Partial marks **will not** be given for incomplete functionality. Late submissions will incur a **10% penalty per day**, rolling over at **12.00 AM**.

Extensions

Familiarise yourself with the assessment due date. Extensions will **only** be granted if you are unable to complete the assessment by the due date because of **unforeseen circumstances outside your control**. The length of the extension granted will depend on the circumstances and **must** be negotiated with the course lecturer before the assessment due date. A medical certificate or support letter may be needed. Extensions will not be granted on the due date and for poor time management or pressure of other assessments.

Resits

Resits and reassessments **are not** applicable in **ID511001: Programming 2**.

Instructions

You will need to submit applications and documentation that meet the following requirements:

Functionality - Learning Outcome 1 (55%)

Part 1 (10%)

- **Institution.cs**. This class should have information such as name, region and country.
- **Department.cs**. This class should have information such as institution and name. **Note:** The institution field should be of type **Institution**.
- **Course.cs**. This class should have information such as department, code, name, description, credits and fees. **Note:** The department field should be of type **Department**.
- **Seeder.cs**. This class should have three methods to seed the lists of **Institution**, **Department** and **Course** objects. Each list should contain at least three objects.

Part 2 (10%)

- **CourseAssessmentMark.cs**. This class should have information such as course and assessment marks. The assessment marks should be a list of integers. This class should also have methods to get all marks, all grades, highest marks, lowest marks, fail marks, average mark and average grade.

For more information on how to calculate the highest, lowest and fail marks, refer to the **grade table** in the **Additional Information** section below.

Part 3 (10%)

- The application needs to contain the following **enums**:

```
public enum EPosition
{
    LECTURER = 0,
    SENIOR_LECTURER = 1,
```

```
    PRINCIPAL_LECTURER = 2,
    ASSOCIATE_PROFESSOR = 3,
    PROFESSOR = 4
}

public enum ESalary
{
    LECTURER_SALARY = 85000,
    SENIOR_LECTURER_SALARY = 100000,
    PRINCIPAL_LECTURER_SALARY = 115000,
    ASSOCIATE_PROFESSOR_SALARY = 130000,
    PROFESSOR_SALARY = 145000
}
```

- **Person.cs**. This class should have information such as id, first name and last name.
- **Learner.cs**. This class should inherit from **Person** and have an additional field for course assessment marks. This field should be of type **CourseAssessmentMark**.
- **Lecturer.cs**. This class should inherit from **Person** and have additional fields for position, salary and course. The position field should be of type **EPosition**, the salary field should be of type **ESalary** and the course field should be of type **Course**.
- **DataReader.cs**. This class should should have two methods to read from the **learners.txt** and **lecturers.txt** files. The first method should read the **learners.txt** file and populate a list of **Learner** objects. The second method should read the **lecturers.txt** file and populate a list of **Lecturer** objects.
- The **learners.txt** file contains the following information:

```
1,Alex,Walker,0,90,95,85,15,5
2,Lucy,Taylor,0,50,40,50,70,60
3,Ethan,Moore,1,60,65,80,85,60
4,Chloe,Adams,1,20,30,40,65,75
5,Noah,Baker,2,80,88,90,50,15
```

- What do the values represent? The first value is the **id**, the second value is the **first name**, the third value is the **last name**, the fourth value is the **course** index (0 for course 1, 1 for course 2, etc.), and the remaining values are the **assessment marks** for that learner. Each learner has five assessment marks.
- The **lecturers.txt** file contains the following information:

```
1,Michael,Brown,1,100000,0
2,Sophia,Green,2,115000,1
3,Ethan,White,0,85000,2
4,Emma,Clark,1,95000,0
5,Liam,Harris,2,110000,1
```

- What do the values represent? The first value is the **id**, the second value is the **first name**, the third value is the **last name**, the fourth value is the **EPosition** enum (0 for LECTURER, 1 for SENIOR_LECTURER, etc.), the fifth value is the **ESalary** enum (85000 for LECTURER, 100000 for SENIOR_LECTURER, etc.), and the sixth value is the **course** index (0 for course 1, 1 for course 2, etc.).

Part 4 (10%)

- **Form1.cs**. This class should have the following functionality:
  - Seed the **institutions**, **departments** and **courses** lists by calling methods from the **Seeder** class.
  - Read the **learners.txt** and **lecturers.txt** files by calling methods from the **DataReader** class.
  - Display the course details when the user clicks the **Display course details** button. The course details should include the course code, name, description, credits, fees, institution name, region and country, and department name.
  - Display all marks when the user clicks the **Display all marks** button. The marks should include the learner id, first name, last name, course code and name, and assessment marks.
  - Display all grades when the user clicks the **Display all grades** button. The grades should include the learner id, first name, last name, course code and name, and assessment grades.
  - Display highest marks when the user clicks the **Display highest marks** button. The highest marks should include the learner id, first name, last name, course code and name, and highest assessment mark(s).
  - Display lowest marks when the user clicks the **Display lowest marks** button. The lowest marks should include the learner id, first name, last name, course code and name, and lowest assessment mark(s).
  - Display fail marks when the user clicks the **Display fail marks** button. The fail marks should include the learner id, first name, last name, course code and name, and fail assessment mark(s).
  - Display average marks when the user clicks the **Display average marks** button. The average marks should include the learner id, first name, last name, course code and name, and average assessment mark.
  - Display average grades when the user clicks the **Display average grades** button. The average grades should include the learner id, first name, last name, course code and name, and average assessment grade.
  - Display lecturer details when the user clicks the **Display lecturer details** button. The lecturer details should include the lecturer id, first name, last name, position, institution name, region and country, department name, course code and name, and salary.
  - Add a learner when the user clicks the **Add learner** button. The user should be prompted to enter the first name, last name, course and assessment marks 1-5. The learner should be added to the **learners** list and written to the **learners.txt** file.
  - Add a lecturer when the user clicks the **Add lecturer** button. The user should be prompted to enter the first name, last name, position and course. The lecturer should be added to the **lecturers** list and written to the **lecturers.txt** file. The salary should be calculated based on the position.
  - **id** is auto-generated and unique for both learners and lecturers. The id should be the next available number in the list.
  - When adding a learner or lecturer, the following validation should be performed:
    - The first name and last name should not be empty, and should not contain numbers or special characters.
    - The course should be selected from the list of courses.
    - The assessment marks should be between 0 and 100.
    - The position should be selected from the list of positions.
  - Remove a lecturer when the user clicks the **Remove a lecturer** button. The user should be prompted to enter the id of the lecturer to remove. If the id is valid, the lecturer should be removed from the **lecturers** list and **lecturers.txt** file.
  - When the user clicks the **Exit** button, the application should close.

Part 5 (15%)

In parts 1-4, you implemented the core functionality. In this part, you will select four additional features from the list below to implement in your application. Each additional feature has been marked with either **Medium** or **Hard**.

Search:

- **Medium** - Search for a learner and lecturer by id, first name or last name. The user should be able to enter a search term and the application should display the matching learners and lecturers.
- **Medium** - Search for a course by code or name. The user should be able to enter a search term and the application should display the matching courses.

Sort and Filter:

- **Medium** - Sort learners and lecturers by id, first name or last name. The user should be able to select the sorting criteria and order (ascending or descending) from a dropdown list.
- **Medium** - Sort courses by code, name or fees. The user should be able to select the sorting criteria and order (ascending or descending) from a dropdown list.
- **Hard** - Filter learners and lecturers by course. The user should be able to select a course from a dropdown list and the application should display the learners and lecturers for that course.
- **Medium** - Filter learners by assessment mark. The user should be able to enter a minimum and maximum mark and the application should display the learners with marks within that range.
- **Medium** - Filter courses by country. The user should be able to select a country from a dropdown list and the application should display the courses for that country.

Statistics:

- **Hard** - Calculate and display the completion rate for each course based on the number of learners who have completed all assessments.
- **Hard** - Calculate and display the average assessment marks and grades for each institution.
- **Medium** - Calculate and display the average salary of all lecturers.
- **Hard** - Calculate and display the total fees collected for each course based on the number of learners enrolled in that course.
- **Hard** - Calculate and display the percentage of learners who have achieved merit (B-range) and distinction (A-range) grades for each course.

Code Quality and Best Practices - Learning Outcome 2 (40%)

- Naming Conventions (5%) - File, class, method, variable, constant and property names should be clear, descriptive and consistent across the codebase.
- Documentation and Comments (10%) - The code should be well-documented using the **XML** documentation format with comments that explain complex logic and clarify the purpose of classes, methods or complex sections.
- Code Formatting (10%) - Consistent indentation and spacing should be used across the codebase. It includes ensuring proper indentation for code blocks, following a standard format for braces and adding blank lines between sections.
- Dead Code (5%) - The code should not contain unused or redundant files, classes, methods, variables, constants or properties. Keeping unnecessary code around can lead to confusion, clutter and potential bugs down the line.
- Performance and Scalability (10%) - The code should be optimised for performance, using efficient algorithms and data structures to minimise bottlenecks.

Class Diagram - Learning Outcome 2 (2.5%)

- Generate a class diagram using Visual Studio's built-in tools. This diagram should include all classes, methods and properties used in the application.

Version Control - Learning Outcome 2 (2.5%)

- Your **Git commit messages** should reflect the context of each functional requirement change.

Additional Information

- You may add additional classes and methods.
- Grade and mark range table:

Grade	Mark Range
A+	90-100
A	85-89
A-	80-84
B+	75-79
B	70-74
B-	65-69
C+	60-64
C	55-59
C-	50-54 (passing assessment marks)
D	40-49 (fail assessment marks)
E	0-39 (fail assessment marks)

- **Do not** rewrite your **Git** history. It is important that the course lecturer can see how you worked on your assessment over time.